MOSTEK®

3870/F8 MICROCOMPUTER SUPPORT

Application Note

F8 Family

SUBROUTINE NESTING AND MULTIPLE INTERRUPT HANDLING

INTRODUCTION

The 3870 and F8 Microcomputer Families are quickly becoming recognized as a cost effective method of placing computing power into types of equipment which couldn't have justified computer control just a short time ago. The falling cost per computer function afforded by advances in Metal-Oxide Semiconductor-Large Scale Integration (MOS—LSI) has brought computer technology and techniques into areas where until now, mechanical controller's, random logic and relay logic predominated. The availability of a large number of Input/Output pins in the 3870 Microcomputer coupled with its minimum system configuration of just one device, makes it an ideal replacement for many previously used control devices. The purpose of this note is to discuss the use and implementation of subroutines and interrupts as they apply to programming an F8 based microcomputer system. The intent of this note is to discuss the use of subroutines and interrupts for the hardware designer who might not be totally familiar with the programming of a computer.

SUBROUTINES

A subroutine is a sequence of computer instructions or mnemonics which can be called or used in several portions of the computer program. The purpose of a subroutine is to reduce the total length of a computer program by consolidating in one portion of the program a sequence of instructions that are used in several different areas of the program. When this subroutine is required the program counter contents are replaced with the starting address of the subroutine. At the end of the subroutine the program is transferred back to the main body of the program.

Figure 1 depicts program flow when using subroutines. The main program calls a subroutine which causes the program counter to be loaded with the address of the subroutine. The last statement of the subroutine causes a return back to the main program flow by retrieving the saved program counter value, forcing a return to the main program flow. The subroutine is called again any place in the main program flow where the sequence of instructions contained in the subroutine is required. Every time the subroutine is called a savings in program length (and ROM size) equal to the length of the subroutine (minus three) is realized compared to a program which doesn't use subroutines. Many times a subroutine will call another subroutine resulting in what is referred to as nested or multi-level subroutines. Nested subroutines in an F8 system will be discussed in this note.

INTERRUPTS

Interrupts are used in a microcomputer system to make it responsive to the device it is controlling.

PROGRAM FLOW WHEN USING SUBROUTINES

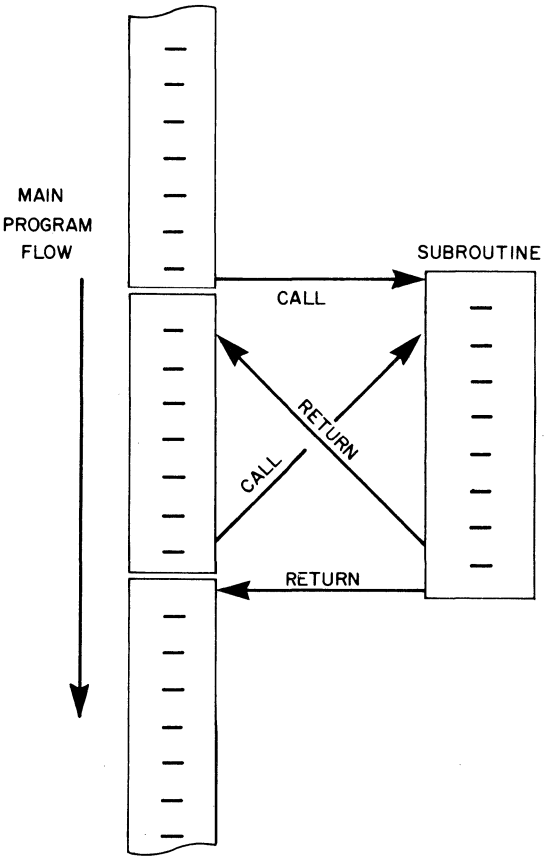

Figure 1

By interrupting the microcomputer the I/O device can signal its requirement for attention or service by the microcomputer. As in the case of the subroutine, the interrupt can divert the main program flow to a sequence of instructions called the Interrupt Service Routine (See Figure 2). This routine either inputs or outputs data to the device being controlled. At the end of this service routine the program counter value at the time of the system interrupt is retrieved from a temporary register and reloaded into the program counter to cause a return to the main

PROGRAM FLOW WHEN INTERRUPTED

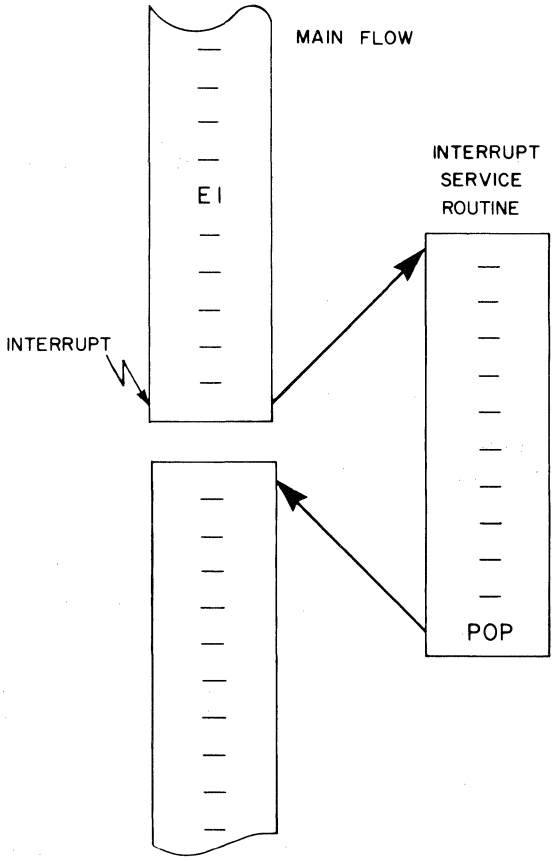

Figure 2

program flow. Interrupts, like subroutines, can be nested because an Interrupt Service Routine could be interrupted by a higher priority device or an Interrupt Service Routine may call a subroutine, which in either case causes nesting.

The F8 instructions which are used to transfer program flow to or from subroutines or interrupts are shown in Figure 3. The Program Counter (P0) holds the address of the next instruction to be executed by the microcomputer while the Stack Register (P)* is a temporary storage location for the Program Counter. In addition two pairs of registers in the Scratchpad have been designated K and Q with instructions that link them to P0 and P. The instructions that link and affect these registers are the following:

(a) Call to subroutine immediate —PI—an instruction which causes the next two bytes in the program to be loaded into the Program Counter (P0) in order to transfer control to a subroutine and saves the old program counter value (return address) in the Stack Register (P).

(b) Call to subroutine—PK—an instruction which causes the contents of the K register to be loaded into the Program Counter while the Program Counter is saved in the Stack Register.

(c) Return from subroutine—POP—an instruction used at the end of a subroutine or interrupt service routine to load the Stack Register back into the Program Counter to return program flow back to the main program. The previous value of the Program Counter is overwritten and lost.

(d) Load—LR P,K—a pair of instructions which
LR K,P
allows the transfer of the Stack Register (P) to the K register in the Scratchpad or vice versa. This switch is useful to save P in preparation for a subroutine or interrupt.

(e) Load — LR P0,Q which allows the transfer of the Program Counter (P0) to the Q register in the Scratchpad.

The following sections of this note will discuss the use of these instructions and registers as well as the general F8 architecture to handle Subroutines, Interrupts and the tradeoffs in doing so.

SUBROUTINES AND/OR INTERRUPTS UP TO TWO LEVELS

Many applications can be handled by two levels of subroutines and/or interrupts. Two levels means that only two return addresses need be saved, which can be handled easily by registers within the F8 for this purpose. The calling of subroutines is under control of the programmer and thus only the return addresses need be saved as other registers (such as the Data Counter) can either be saved by the calling or the called routines if the registers are needed by the subroutine. Interrupts are under control of the programmer only to the extent that they can be masked or enabled. Assuming interrupts are enabled, upon entry to an Interrupt Service Routine, it may not be known which registers in the CPU contain data which cannot be overwritten. In this case these registers should be stored in the scratchpad during the Interrupt Service Routine and be restored before exiting this routine. Examples of using the ISAR to store CPU registers in a push down stack are given in this note but in many cases the programmer will tailor the status saving routine for the specific circumstances of his sytem design (by using specific Scratchpad Registers to save CPU Registers).

Figure 4 shows the instructions usually used to call a subroutine (one level deep) in an F8 system. SUBA1 is the symbolic name of the two byte address of the subroutine and PI causes the return address (XXXX) to be saved in P. POP reverses the procedure at the end of the subroutine causing P0 to be reloaded with the address saved in P causing the program flow to return to the next instruction in the main flow (XXXX). Response to an interrupt from the main flow is similar to this example except that the interrupt causes a path similar to 1 to the Interrupt Service Routine with the address (vector) being supplied by the interrupting device and loaded into P0.

To call a second subroutine or to respond to an interrupt from SUBA1 the instructions in Figure 5 could be used. In this case PI SUBA1 transfers the

* Earlier versions of F8 literature labled the Program Counter PC0 and the Stack Register PC1.

F8 REGISTERS USED IN SUBROUTINES AND INTERRUPTS

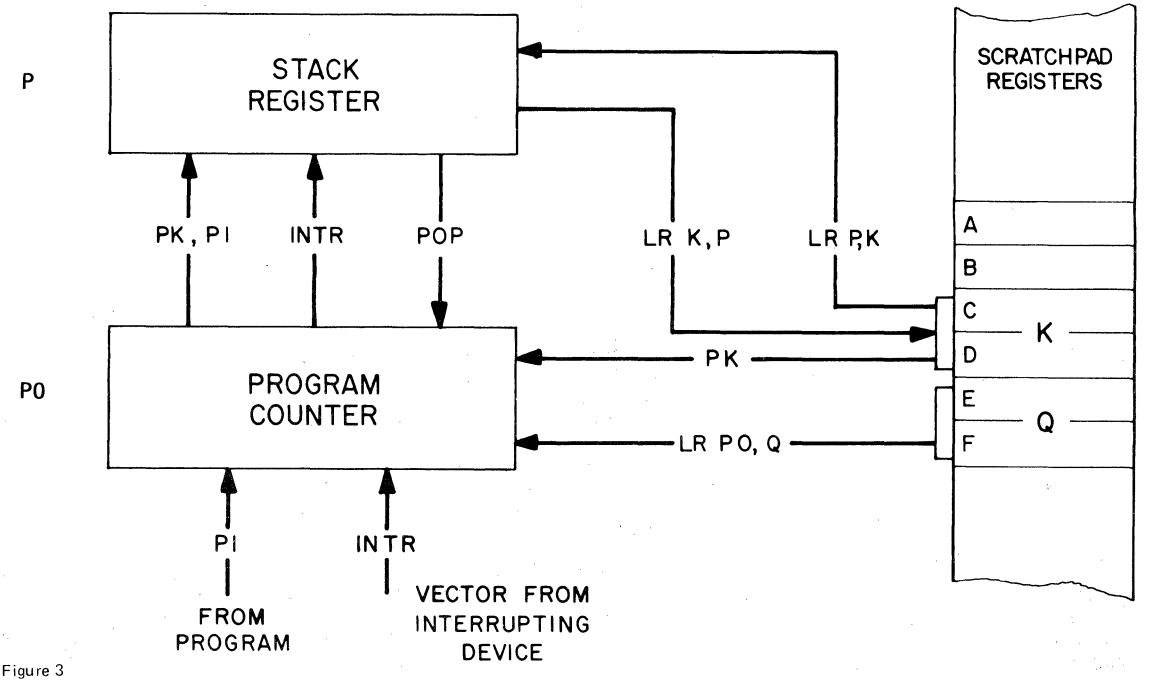

Figure 3

ONE LEVEL SUBROUTINES OR INTERRUPTS

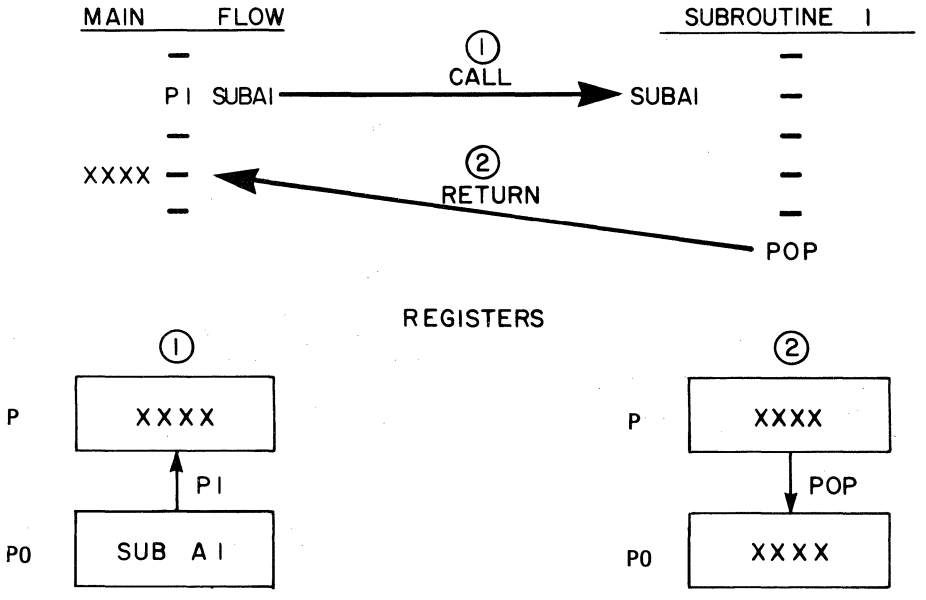

Figure 4

program flow to SUBA1 while saving the return address (XXXX) in the Stack Register (P). Subroutine 1 now transfers P to K in preparation for another subroutine or an interrupt (note that if an interrupt occurs during the PI SUBA1, LR K,P sequence it will not be serviced until after the LR K,P instruction because PI is privileged). Subroutine 2 is called by PI SUBA2 which saves YYYY in P which was just vacated. Program flow transfers to Subroutine 2 and the POP instruction reloads P0 with YYYY from P. At the end of Subroutine 1 the return address is now in K so a PK is used to load XXXX into P0, thereby returning to the main program. Note that LR K,P followed by POP could have been used in place of the PK instruction, but would be 1 byte longer.

Three levels of subroutines or interrupts can be handled by using the Q register to save a return address. Figure 6 shows programming with three levels of subroutines (three levels of interrupts would be handled in the same manner). The first subroutine is called from the main program and the return address is saved in the Stack Register P. At the beginning of SUB1, P is transferred to the K register in preparation for the second subroutine call or interrupt. This second call uses the just vacated P register for storage of the return address to SUB1. Upon entering SUB 2 both P and K contain valid return addresses so that interrupts must be disabled while the contents of K register are moved to the Q register and the contents of P are moved to the K register. Once P is clear, interrupts can be enabled by the use of the EI instruction, allowing a third level of subroutine nesting (as shown in Figure 6) or an interrupt. The return from SUB3 is accomplished by executing the POP instruction which loads the Program Counter with the value RTN2 from the Stack Register P. During the first portion of SUB2, P was moved to K so that a PK instruction will load the Program Counter with RTN1 from the K register. The return address for SUB1 (RTN) was moved to the Q register during the first portion of SUB2 and can be transferred to the Program Counter by the execution of LR P0,Q instruction.

MULTILEVEL INTERRUPTS OR SUBROUTINES

At a minimum when using the F8 in a system with greater than 3 levels of interrupts or subroutines a consistent method of placing return address into the scratchpad must be used to allow their recovery. In many cases it will be desirable to stack more registers than just the return addresses. Previous examples have shown 3 levels deep with the three return addresses in P, K, and Q registers. Any further nesting would destroy either P, K, or Q so the technique to be described is to move K into the scratchpad to make room for another level.

Figure 7 shows a generalized subroutine which automatically transfers P to K and then K into the scratchpad registers. The routines in Figure 7 assume that ISAR (Indirect Scratchpad Address Register) has been initialized at an odd value, probably H'3F'which

TWO LEVEL SUBROUTINES OR INTERRUPTS

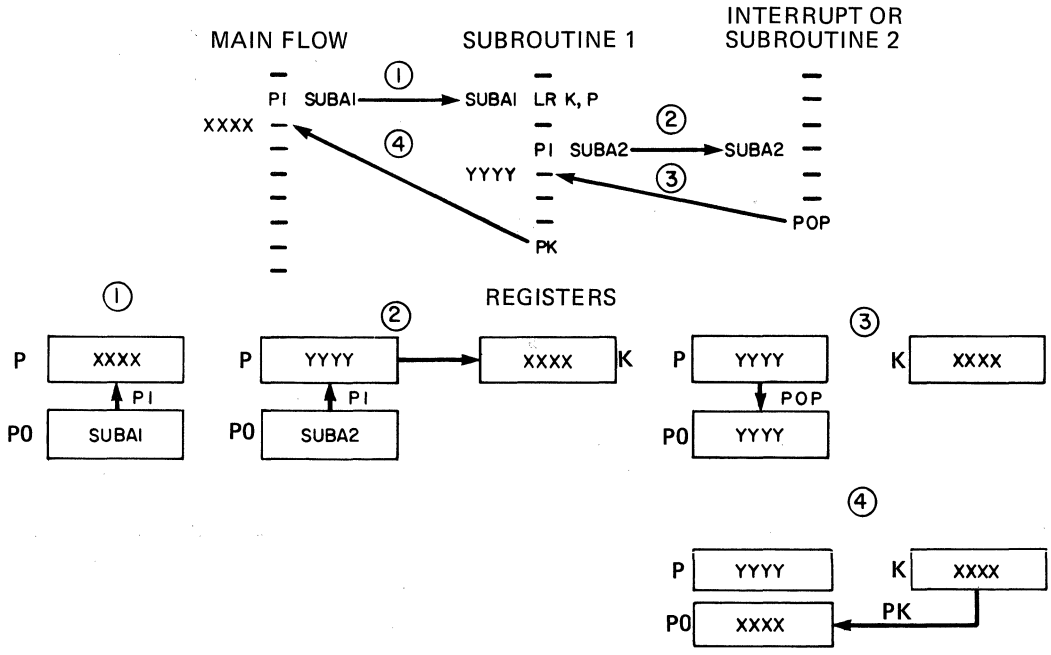

Figure 5

THREE LEVELS OF SUBROUTINES OR INTERRUPTS

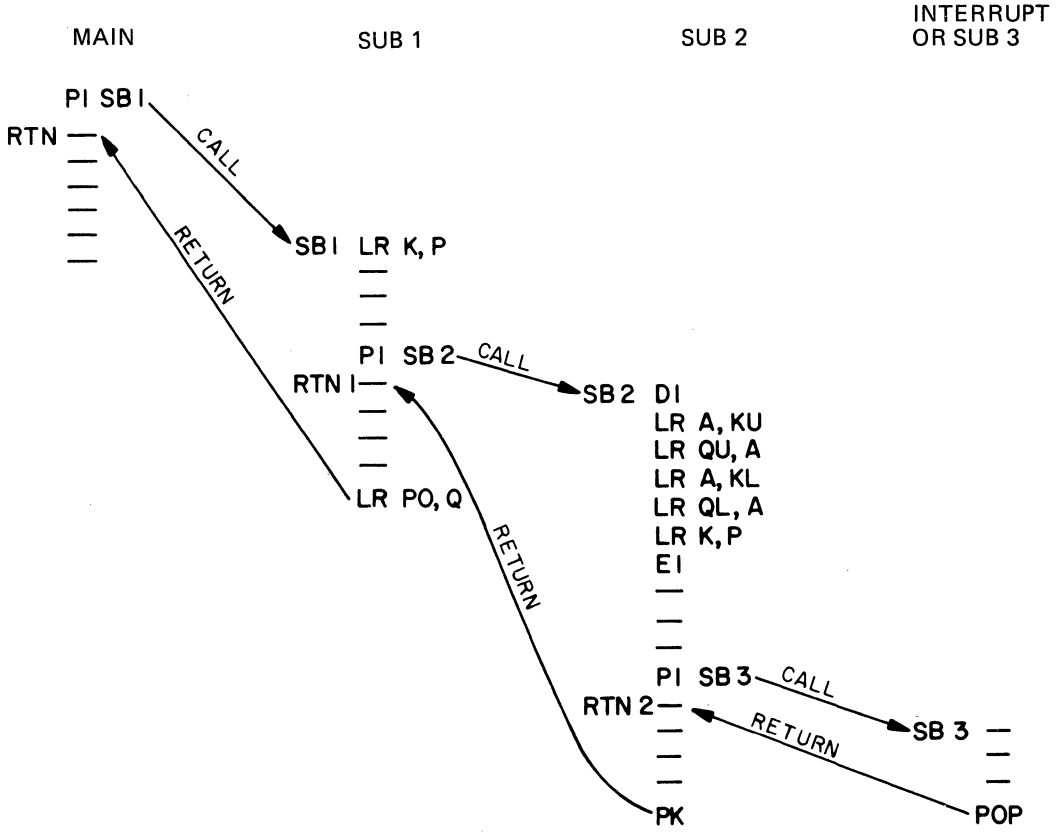

Figure 6

STACKING OF ACCUMULATOR AND STATUS REGISTERS

```
SUBX   DI          }
       LR K,P      } K to
       PI  PSHK    } Stack
       ________    }
       ________    } Body of
       ________    } Subroutine
       ________    }
       PI POPK     } K from
                   } Stack
       PK          }
```

TO STACK

```
PSHK   LR  A, KL
       LR  D, A     KL to stack
       LR  A,KU
       LR  S,A      KU to stack
       LR  A,IS
       AI  H'FF'    Decrement ISAR
       LR  IS, A
       LR  W,J
       POP          Return
```

FROM STACK

```
POPK   LR  A, IS
       INC          Increment ISAR
       LR  IS,A
       LR  A,I
       LR  KU,A     Recover KU
       LR  A,S
       LR  KL,A     Recover KL
       LR  W,J
       POP          Return
```

Figure 7

F8 Family

is the top of the scratchpad registers. The odd starting value is required to insure that ISAR is not pointing to an 8 byte buffer boundary when the LR D, A instruction is executed. (The LR D,A instruction loads the accumulator from the scratchpad location pointed to by ISAR and then does a modulo 8 decrement of ISAR. This means that only the lower three bits of ISAR are decremented resulting in an 8 byte range for these auto decrementing and autoincrementing instructions which does not allow crossing of page boundarys. By initializing ISAR at an odd value, every time the LR D,A instruction is executed ISAR will be odd and therefore will not have to cross page boundaries which are even.) The decrement from even values is accomplished by loading ISAR into the accumulator and adding hexidecimal FF to it, which results in an 8 bit decrement of ISAR's contents. PSHK then moves the contents of K onto the stack and leaves ISAR pointing to the next empty location thus implementing a push-down stack. When in the body of the subroutine both P and K are clear, allowing a call to another subroutine of this format or the enabling of interrupts to allow interrupting out of this subroutine (return address would be held in P). POPK is called to recover the subroutine return address and place it in the K register. Note here that the 8 bit increment (INC) is done first to cross the page boundary and point to the last byte stored on the stack. ISAR is used to pull the return address off the stack and place it in the K register. The PK instruction reloads P0 and program flow is returned to the calling routine.

If interrupts are enabled during the body of the program they must be disabled during execution of POPK because this routine is using both P and K registers. The LR W,J instruction allows the user the option to control whether interrupts will be enabled or disabled after execution of PSHK or

SUBROUTINE OR INTERRUPTS NESTED 4 LEVELS DEEP

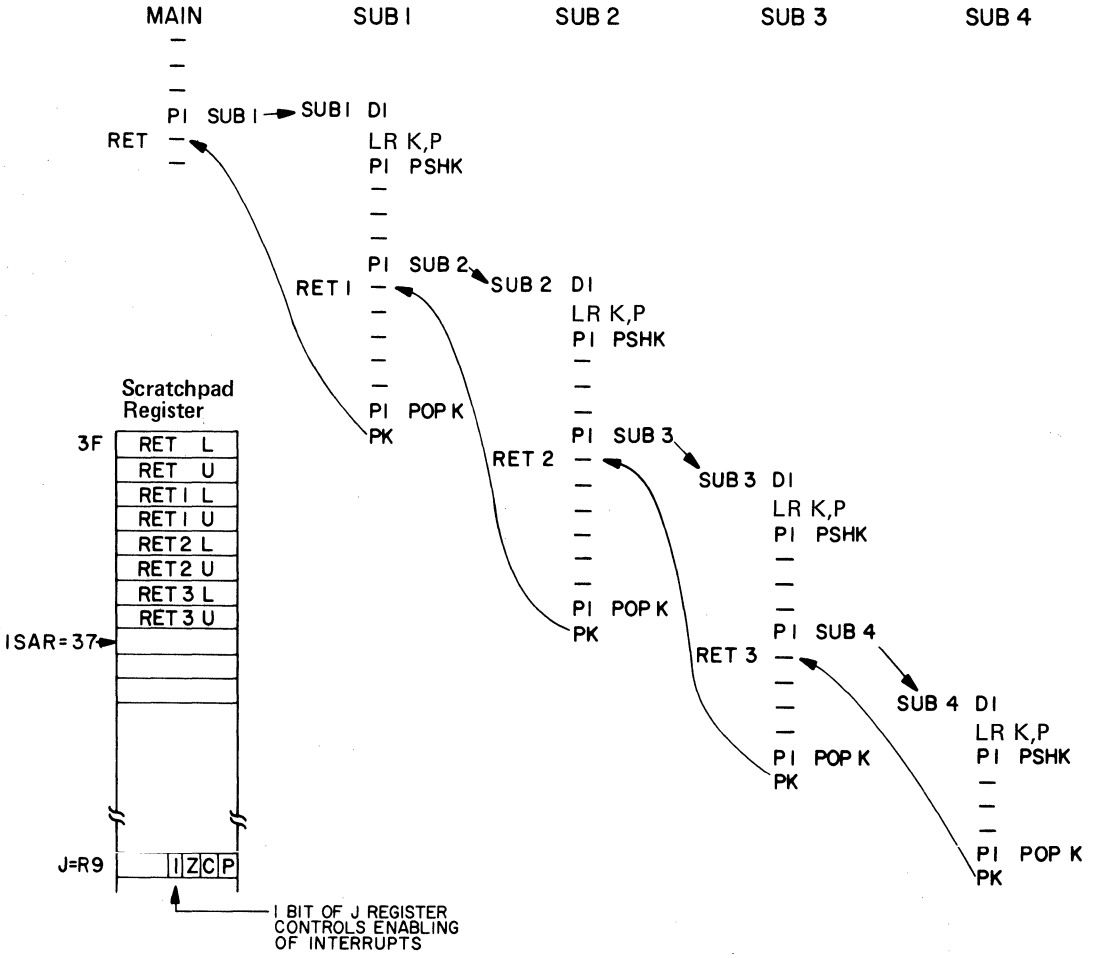

Figure 8

POPK by setting or clearing bit 4 in the J register. Thus if interrupts are desired during a subroutine the following instructions would be used to call SUBX.

MAIN PROGRAM

```
LR  A,IS
LR  0,A    SAVE ISAR IN R0
LI  H'09'
LR  IS,A   POINT ISAR TO J
LR  A,S    GET J INTO A
OI  H'10'  SET 1 BIT, INTERRUPTS ENABLE D
LR  S,A    A INTO J
LR  A,0    RESTORE ISAR
LR  IS,A
PI  SUBX   CALL SUBROUTINE
```

If interrupts are not desired during the nesting of subroutines the software can be simplified as follows: Assuming interrupts are disabled instructions DI and LR W,J can be deleted from the PSHK and POPK routines. Also the above calling sequences will not be required because bit 4 in the W register will already be clear.

Figure 8 shows the effects of PSHK used to save the return address on the stack. Note that the Stack Pointer (ISAR) is pointing to the next empty location on the stack and that bit 4 of the J register is controlling whether interrupts are enabled or not through the use of the LR W,J instruction in PSHK and POPK. Bit 4 of the J register is used to control whether interrupts are enabled or not in order to allow two different callers to use this subroutine. If one of the callers was in an interrupt driven portion of the program, bit 4 of the J register could be set to allow interrupts upon returning from the subroutine. If one of the other callers of the subroutine did not want interrupts enabled bit 4 of the J register could be cleared so that no interrupts would be recognized upon returning from the subroutine.

At the end of each subroutine, PI POPK followed by PK will be executed to unload the stack and return program flow back to the correct return address. Note also that interrupts will be allowed at any time except during the execution of the PSHK or POPK routines, their return address being stored in the P register. If the interrupting device service routine needs to call a subroutine, P will have to be pushed onto the stack using the methods previously described.

In many cases it may be desirable to save the major registers within the CPU whenever an interrupt is serviced. Saving registers on the stack frees up all of the computing power of CPU for use by the Interrupt Service Routine. Saving the registers upon the stack rather than in direct scratchpad locations makes the subroutine or interrupt service routine re-entrant (i.e., the routine calls itself without destroying scratch locations). The same philosophy as before can be used to save accumulator and status register on the stack (see Figure 9). Other registers within the machine could be saved using the technique; however, they must be pushed in pairs in order to leave ISAR pointing to an odd register location (since K, DC and DC1 are all 16 bit registers this should not be a limitation).

STACKING OF ACCUMULATOR AND STATUS REGISTERS

```
PSHAW   LR  D,A     Accm to stack
        LR  A,J
        LR  S,A     J Reg to stack
        LR  J,W     W reg to J reg
        LR  A,IS    ISAR to A
        AI  H'FF'   Decrement A
        LR  IS,A    A to ISAR

POPAW   LR  A, IS   ISAR to A
        INC         Increment A
        LR  IS,A    A to ISAR
        LR  W,J     J reg to W
        LR  A,I     J from stack
        LR  J,A     into J reg
        LR  A,S     Accm from stack
        POP         Return to caller
```

Figure 9

CONCLUSION

This application note has discussed a general method of handling subroutines and interrupts in an F8 system. Many applications for which the F8 is suited will have minimal subroutines or a minimum number of interrupts so that the internal P and K registers can be used to hold return addresses.

In the cases where deeply nested subroutines or multiple interrupts must be handled a push-down stack can be created in the scratchpad registers or external memory. Software routines were discussed to save return addresses in this stack as well as methods to save the general purpose registers. The user has the option in a F8 system to stack only what is necessary to accomplish his design goals in an optimum manner.